

1-Day Technical Challenge: MERN Task Manager

Role: MERN Stack Developer

Time Limit: 1 Day

1. Objective

Build a simple **Task Management App**.

Admin: Can create users and assign tasks to them.

Developer: Can see their assigned tasks and mark them as "Completed".

2. Tech Stack

Frontend: React.js, Tailwind CSS

Backend: Node.js, Express.js

Database: MongoDB, Mongoose

Validation: Joi, Zod

Phase 1: The Backend (API)

Keep it simple. Focus on getting the data to flow.

A. Database Models (Schema)

You only need two schemas:

1. **User:** **name**, **email**, **password**, **role** ('admin' or 'dev').
2. **Task:** **title**, **description**, **dueDate**, **status** ('pending', 'completed'), **assignedTo** (User ID).

B. Required API Endpoints

Do not worry about versioning (**/v1**) for now.

1. Auth

POST /register: Create a new user.

POST /login: Authenticate and return a token/JWT.

2. Users

GET /users: (Admin Only) Returns a list of all users.

Why: The Admin needs this list to populate a dropdown menu when assigning a task to a specific developer.

3. Tasks

POST /tasks: (Admin Only) Create a task and assign it to a user ID.

GET /tasks: Get tasks based on role.

If Admin: Returns **all** tasks.

If Dev: Returns **only tasks assigned to me**.

PUT /tasks/:id: (Dev Only) Update status to 'Completed'.

DELETE /tasks/:id: (Admin Only) Delete a task.

Why: Allows the Admin to remove tasks created by mistake or cancelled work.

Phase 2: The Frontend (React)

Don't spend too much time on design. Use Tailwind to make it clean, but functionality comes first.

Required Pages

1. Login / Register

Simple form to log in.

Save the JWT token in **localStorage** to manage sessions.

2. Dashboard (The Main View)

If logged in as Admin:

Create Task: A form with a dropdown to select a Developer (populated by **GET /users**).

All Tasks: Show a list of all tasks.

Delete: A button to delete a task.

If logged in as Developer:

My Tasks: Show a list of tasks assigned only to the logged-in user.

Complete: A button on each task to mark it as "Completed".

3. Submission Instructions

Priorities:

1. **Functionality:** Can I log in? Can I create a task? Does the Developer see *only* their own tasks?
2. **Code Cleanliness:** Is the code easy to read? (Use meaningful variable names).

Deliverables:

A GitHub link with the code.

A short **README.md** telling us how to start the app (e.g., **npm run dev**).

Best of Luck!